

# Software Requirements Specification (SRS)

## Hostel Management System

*Requirements documentation aligned with ISO/IEC/IEEE 29148:2018*

|                         |                                   |
|-------------------------|-----------------------------------|
| <b>Project</b>          | Hostel Management System          |
| <b>Document Version</b> | 1.0                               |
| <b>Prepared for</b>     | Software Engineering Mini-Project |

## 1. Introduction

### 1.1 Purpose

This Software Requirements Specification (SRS) defines the functional, non-functional, and security requirements for the Hostel Management System. The document establishes a verifiable baseline for design, implementation, and testing.

### 1.2 Scope

The Hostel Management System manages student details, hostel room details, room allocation, room deallocation, and occupancy information. The system is intended for use by a hostel administrator/warden and, within the defined scope, allows a student to view the student's own allocation information. The system does not define fee collection, mess management, visitor management, attendance, or complaint management.

### 1.3 Definitions and Abbreviations

| Term                 | Definition                                                                            |
|----------------------|---------------------------------------------------------------------------------------|
| FR                   | Functional requirement.                                                               |
| NFR                  | Non-functional requirement.                                                           |
| RBAC                 | Role-based access control.                                                            |
| Allocation           | Association between a student and an available hostel room.                           |
| Administrator/Warden | Authorized hostel staff member who manages student, room, and allocation information. |

### 1.4 References

The requirements structure and quality criteria are based on [1]. Testing and architecture documentation referenced by this project are documented separately in accordance with [2] and [3].

### 1.5 Document Overview

Section 2 provides the overall product description. Section 3 specifies the functional and non-functional requirements. Section 4 defines security objectives and requirements. Section 5 provides traceability between requirements and design/test artifacts.

## 2. Overall Description

### 2.1 Product Perspective

The system is a web-based application composed of a client interface, an application/API layer, application services, a data-access layer, and a relational database. The architecture is technology-neutral so that the application may be implemented using one of the technologies listed for the project (for example, Django/Flask or Laravel).

## 2.2 Product Functions

- Authenticate authorized users and enforce role-based access.
- Create, view, search, and update student records.
- Create, view, and update hostel room records and room capacity.
- Allocate an available room to a student while preventing invalid allocations.
- Modify or deallocate an existing room allocation.
- Display current room occupancy and student allocation information.
- Allow a student to view the student's own allocation information.

## 2.3 User Classes

| User class                    | Responsibilities                                                                                                              |
|-------------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| Hostel Administrator / Warden | Maintains student and room records; performs allocation and deallocation; views occupancy; handles administrative operations. |
| Student                       | Views own student/allocation information after successful authentication.                                                     |

## 2.4 Operating Environment

The system shall operate as a browser-accessible web application. The implementation language and framework are not fixed by the project statement; the design therefore specifies technology-independent interfaces and behavior.

## 2.5 Assumptions and Constraints

- Each student has a unique student identifier within the hostel system.
- Each room has a unique room identifier and a configured capacity greater than zero.
- A student shall have at most one active room allocation at a time.
- Only authorized hostel staff may create or modify room allocation records.
- The relational database provides transactional support for allocation updates.
- The measurable NFR targets in Section 3.2 are project targets introduced to make the requirements verifiable; they are not stated as existing system performance claims.

## 2.6 UML Use Case Diagram

Figure 1 shows the principal actors and use cases within the system boundary. Associations represent the interactions included in the project scope.

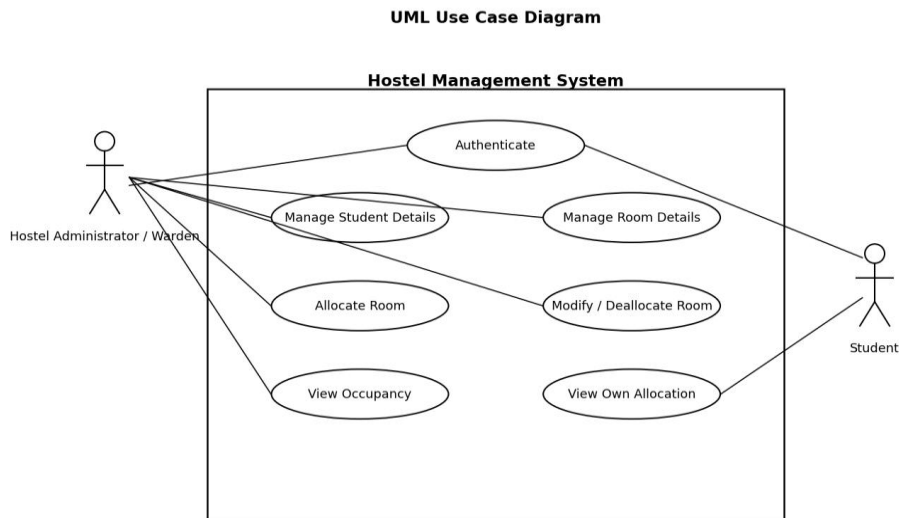


Fig. 1. UML use case diagram for the Hostel Management System.

### 3. Specific Requirements

#### 3.1 Functional Requirements

| ID    | Requirement                                                                                                                                                                                                       | Verification                                                  |
|-------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------|
| FR-01 | The system shall authenticate a user using a registered identifier and password before permitting access to protected functions.                                                                                  | Verifiable by authentication tests.                           |
| FR-02 | The system shall allow an authorized hostel administrator/warden to create and update student records containing the student identifier, name, and current hostel-related details required by the implementation. | Required fields and update behavior can be tested.            |
| FR-03 | The system shall allow an authorized hostel administrator/warden to search for a student record using the student identifier and shall display the matching record.                                               | Search input/output is observable.                            |
| FR-04 | The system shall allow an authorized hostel administrator/warden to create and update room records containing a unique room identifier and room capacity.                                                         | Room creation/update is observable.                           |
| FR-05 | The system shall allocate a selected student to a selected room only when the student is eligible for allocation, the room exists, and the room has available capacity.                                           | Positive and negative allocation cases are testable.          |
| FR-06 | The system shall reject an allocation when the selected room has no available capacity or when the student already has an active allocation.                                                                      | Conflict conditions are testable.                             |
| FR-07 | The system shall allow an authorized hostel administrator/warden to modify or deallocate an active room allocation.                                                                                               | Allocation state transitions are testable.                    |
| FR-08 | The system shall display current occupancy information for rooms, including occupied and available capacity.                                                                                                      | Displayed values can be compared with stored allocation data. |
| FR-09 | The system shall allow an authenticated student to view the student's own allocation information and shall not expose another student's allocation through the student interface.                                 | Authorization and data-isolation tests are testable.          |

#### 3.2 Non-Functional Requirements

| ID     | Attribute       | Requirement                                                                                                                                                                                       | Verification             |
|--------|-----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------|
| NFR-01 | Performance     | For a test workload of up to 50 concurrent authenticated users, at least 95% of normal read requests shall complete within 2 seconds, excluding network latency outside the application boundary. | Performance test         |
| NFR-02 | Availability    | During scheduled service hours, the deployed system shall provide at least 99% monthly availability, excluding planned maintenance announced in advance.                                          | Operational measurement  |
| NFR-03 | Data integrity  | A room allocation operation shall be atomic: either the allocation and associated capacity update are both committed, or neither change is committed.                                             | Transaction/failure test |
| NFR-04 | Usability       | A trained administrator shall be able to complete a valid room allocation using no more than five primary data-entry steps after login.                                                           | Usability test           |
| NFR-05 | Maintainability | The implementation shall separate presentation, application-service, and data-access responsibilities so that each layer can be modified without changing the interface of unrelated layers.      | Design inspection        |

### 3.3 Requirement Quality Criteria

Every requirement in this SRS is written as a testable statement with a unique identifier. Requirements use the term “shall” for mandatory behavior and are intended to be clear, unambiguous, concise, consistent, feasible, and verifiable, consistent with the requirements-engineering guidance in [1].

## 4. Security Requirements

### 4.1 Security Objectives

| ID    | Security objective                                                                                         |
|-------|------------------------------------------------------------------------------------------------------------|
| SO-01 | Prevent unauthorized access to protected student, room, and allocation information.                        |
| SO-02 | Preserve the confidentiality and integrity of student and allocation data during storage and transmission. |

### 4.2 Security Requirements

| ID     | Security requirement                                                                                                                                                                          |
|--------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| SEC-01 | The system shall require successful authentication before a user can access protected API endpoints or protected application functions.                                                       |
| SEC-02 | The system shall enforce role-based authorization so that student users can access only permitted student-facing functions and cannot create, modify, or delete room allocation records.      |
| SEC-03 | The system shall store passwords using a salted, computationally strong password-hashing mechanism and shall not store plaintext passwords.                                                   |
| SEC-04 | The system shall protect authenticated application traffic using HTTPS/TLS in deployed environments.                                                                                          |
| SEC-05 | The system shall validate user-supplied identifiers and fields and shall use parameterized database operations or equivalent ORM protections to prevent injection through application inputs. |
| SEC-06 | The system shall record security-relevant events, including successful and failed authentication and allocation create/modify/deallocate operations, with a timestamp and actor identifier.   |

## 5. Requirements Traceability

| Req. ID | Requirement area                          | Security linkage | Test / verification |
|---------|-------------------------------------------|------------------|---------------------|
| FR-01   | Authentication                            | SEC-01, SEC-02   | TP-01, TP-10        |
| FR-02   | Student record management                 | SEC-05           | TP-02               |
| FR-03   | Student search                            | SEC-05           | TP-03               |
| FR-04   | Room management                           | SEC-05           | TP-04               |
| FR-05   | Room allocation                           | SEC-06           | TP-05, TP-06        |
| FR-06   | Invalid/conflicting allocation prevention | SEC-02, SEC-06   | TP-06, TP-07        |
| FR-07   | Modify/deallocate allocation              | SEC-02, SEC-06   | TP-08               |
| FR-08   | Occupancy view                            | SEC-05           | TP-09               |
| FR-09   | Student self-view                         | SEC-02           | TP-10               |
| NFR-01  | Performance                               | -                | TP-11               |

| Req. ID | Requirement area      | Security linkage | Test / verification      |
|---------|-----------------------|------------------|--------------------------|
| NFR-02  | Availability          | -                | Operational verification |
| NFR-03  | Transaction integrity | SEC-06           | TP-07, TP-12             |
| NFR-04  | Usability             | -                | TP-13                    |
| NFR-05  | Layer separation      | -                | Architecture inspection  |

## References

- [1] ISO/IEC/IEEE 29148:2018, \*Systems and software engineering — Life cycle processes — Requirements engineering\*, IEEE, 2018.
- [2] ISO/IEC/IEEE 29119-3:2021, \*Software and systems engineering — Software testing — Part 3: Test documentation\*, IEEE, 2021.
- [3] ISO/IEC/IEEE 42010:2022, \*Software, systems and enterprise — Architecture description\*, IEEE, 2022.
- [4] Software Engineering Mini-Project Deliverables Part-1, course handout, September 2026.